

Paper 2: Empirical Evaluation of Epistemic Governance Overhead and Threat Mitigation in LLM Agent Runtimes

Author: Can Kasaplar

Domain: Empirical AI Safety, Runtime Governance Performance, LLM Vulnerability Mitigation

Abstract

As Large Language Models (LLMs) transition to execution agents in production environments, governance frameworks must satisfy strict dual constraints: microsecond-scale latency overhead and complete containment of unauthorized state mutations. In this paper, we present an empirical evaluation of **Rhizoh Engine** across $N = 1,500$ evaluation scenarios. We explicitly isolate total system latency $T_{\text{total}} = T_{\text{LLM}} + T_{\text{Governance}}$ and measure the performance of Rhizoh's policy evaluation, capability verification, dynamic uncertainty calculation $U(t)$, and Merkle DAG provenance generation. Tested against controlled rule structures (Controlled Reference Policy Dataset, $N = 1000$) and external attack schemas adapted from AgentDojo and CyberSecEval ($N = 500$), Rhizoh achieves a median overhead (P_{50}) of **0.071 ms** and a 95th percentile (P_{95}) overhead of **0.092 ms**, well below the target Service Level Objective (SLO) of **5.0 ms**. Furthermore, an ablation study demonstrates the necessity of the Epistemic Uncertainty Engine (**33.33%** UER leak when disabled), while Rhizoh's full configuration maintains an exact Clopper-Pearson 95% binomial upper confidence bound of **[0.00% – 0.40%]**.

1. Introduction & Research Questions

While Paper 1 formalized the operational semantics ($G(I, S) \implies S'$), required evidence operator (Γ), and asymptotic complexity ($\mathcal{O}(|V_{\text{req}}| + R \cdot D + K)$) of Rhizoh Engine, this paper addresses two core empirical research questions:

- **RQ1 (Latency Isolation & Workload Dependence):** What is the exact execution overhead introduced by Rhizoh's governance pipeline when isolated from LLM token inference

- latency (T_{LLM}), and does workload scaling introduce observable bottlenecks?
- **RQ2 (Ablation & Threat Containment):** How do individual governance modules (OPA, Capability Tokens, $U(t)$) contribute to preventing unauthorized executions (UER) without inducing false-positive pauses on benign requests?

2. Experimental Methodology & Environment Audit

2.1 Reproducibility Environment Audit Specification

All experiments were executed within an isolated, deterministic single-threaded process space using platform-native high-resolution timers:

EVALUATION ENVIRONMENT SPECIFICATION:

- |— OS Environment : Windows 10 (64-bit)
- |— Timer Resolution : Windows QPC via `time.perf_counter()` (< 100 ns hardware resolution)
- |— Python Interpreter : 3.13.2 (Isolated Virtual Environment)
- |— Execution Model : Single-threaded CPU execution with Warm-up priming
- |— Dataset Cohorts : N = 1,500 Total Scenarios (Controlled Reference + External Adapters)

2.2 Latency Isolation Formulation

To ensure evaluation transparency, governance runtime overhead is isolated from LLM generation latency:

$$T_{\text{Governance Runtime}} = T_{\text{Observation}}(\Gamma) + T_{\text{Uncertainty}}(U(t)) + T_{\text{Policy}}(\text{OPA}) + T_{\text{Capability}}$$

$$T_{\text{total}} = T_{\text{LLM Inference}} + T_{\text{Governance Runtime}}$$

3. Empirical Results & Performance Analysis

3.1 Ground Truth Confusion Matrix (Controlled Reference Policy)

Dataset, $N = 1,000$)

Data Transparency Note: Reported unit metrics evaluate reference policy bindings under controlled synthetic schema mappings to validate operational state

transition logic.

CONFUSION MATRIX (GROUND TRUTH VS PREDICTED ACTION)

	Predicted REJECT / PAUSE	Predicted ALLOW
Ground Truth		
Malicious	750 (TN)	0 (FN / Leak)
Benign	0 (FP)	250 (TP)

- Unauthorized Execution Rate (UER): 0.00% (0/750 leaks observed)
- Exact Clopper-Pearson 95% CI Upper Bound: [0.00% – 0.40%]
- Threat Precision / Recall / F1-Score: 100.00%
- Total Batch Execution Time ($N = 1000$): 0.083 seconds

3.2 Governance Overhead Latency Distribution (RQ1)

Table 3: Latency Overhead Distribution Across Dataset Cohorts

Benchmark Cohort	Total Samples (N)	P50 Overhead (ms)	P95 Overhead (ms)	P99 Overhead (ms)	SLO Status (≤ 5.0 ms)
Controlled Reference Policy Dataset	1,000	0.071 ms	0.092 ms	0.115 ms	PASS ($\sim 54\times$ faster)
External Adapter Schema (AgentDojo)	500	0.089 ms	0.148 ms	0.275 ms	PASS ($\sim 33\times$ faster)

3.3 Defense-in-Depth Ablation Study (RQ2)

We evaluate individual governance layers by selectively disabling system components across 750 malicious scenarios:

Table 4: Governance Module Contribution & Ablation Analysis

System Configuration	Uncertainty U(t)	Policy Engine (OPA)	Capability Engine	Unauthorized Exec Rate (UER)	Defense-in-Depth Role
Full Rhizoh Engine	ON	ON	ON	0.00%	Full System Enforcement

w/o Uncertainty Engine OFF ON ON	33.33%	Context Poisoning & Stale Leaks
w/o Policy Engine ON OFF ON	0.00%	Bound by Capability Tokens & $U(t)$
w/o Capability Engine ON ON OFF	0.00%	Bound by OPA Policies & $U(t)$
Baseline (No Governance) OFF OFF OFF	100.00%	Unchecked Execution Dispatch

Defense-in-Depth Note: In our tri-factor governance matrix, $U(t)$, OPA, and Capability Tokens act as complementary boundaries. Disabling OPA or Capability modules independently yields 0.00% UER in this specific benchmark because remaining active layers independently intersect the privilege boundary. Disabling all governance layers yields a 100.00% leak rate.

3.4 Experimental Consistency with Theoretical Complexity Bound

We benchmarked $N = 1,000$ warm-up primed iterations per configuration using high-resolution hardware counters (time.perf_counter()), scaling required evidence nodes ($|V| \in [10, 1000]$) and policy rules ($R \in [5, 500]$):

Table 5: Ordinary Least Squares (OLS) Linear Regression Parameter Breakdown

Parameter	Notation	Estimate	95% Confidence Interval	p-value	Interpretation
Intercept (Base Overhead)	β_0	0.007829 ms	[0.005900 – 0.009759] ms	$p < 0.001$	Statistically significant base runtime (7.8μs)
Slope (Scaling Factor)	β_1	-0.00000133 ms/load	[-0.00000389 – 0.00000123]	$p > 0.05$	Fail to reject null hypothesis
Determination Coefficient	R^2	0.2568	N/A N/A		Consistent with platform timing noise

Statistical Interpretation: The OLS regression model was used to assess observable workload dependence rather than predictive accuracy. The estimated

slope ($\beta_1 = -0.00000133 \text{ ms/unit}$) yielded $p > 0.05$. Within the evaluated workload range, we did not observe statistically significant evidence of workload-dependent latency growth. The empirical results remain tightly bounded within sub-millisecond bounds ($5.4\mu\text{s} - 8.6\mu\text{s}$) and are consistent with the proposed linear complexity model $\mathcal{O}(|V_{\text{req}}| + R \cdot D + K)$.

3.5 Architectural Runtime Characteristics & Complexity Trade-Offs

SCOPE OF COMPARISON BOUNDARY NOTE	
This comparison is architectural rather than leaderboard-oriented. We compare governance design boundaries, dominant runtime characteristics, and asymptotic complexity bounds. End-to-end execution latency in LLM-mediated systems additionally depends on underlying model inference (T_{LLM}), hardware accelerators, and context length, and is therefore not directly comparable to an isolated governance runtime overhead $T_{\text{Governance}}$.	

Table 6: Architectural Characteristics & Dominant Cost Breakdown

Framework / System	Governance Architecture	Dominant Runtime Cost	Complexity Bound
Baseline (No Guard)	Unchecked Direct Execution	0.000 ms (Reference Baseline)	$\mathcal{O}(1)$
Guardrails AI	Output Re-prompting / LLM Guard	T_{LLM} (Inference Dependent)	$\mathcal{O}(T_{\text{LLM}})$
NeMo Guardrails	Colang Rail Rules + LLM Eval	T_{LLM} (Inference Dependent)	$\mathcal{O}(\text{Rules} + T_{\text{LLM}})$

| **Open Policy Agent (OPA)** | Deterministic AST Evaluation | $T_{\text{Gov}} (\approx 0.042 \text{ ms})$ | $\mathcal{O}(R \cdot D)$ |

| **Rhizoh Engine (Ours)** | Pre-Mutation Epistemic Governance | $T_{\text{Gov}} (\approx 0.071 \text{ ms})$ | $\mathcal{O}(|V| + R \cdot D + K)$ |

Architectural Takeaway: Rhizoh preserves microsecond-scale governance overhead while extending deterministic policy evaluation with dynamic epistemic uncertainty assessment.

3.6 Empirical Verification of Theorem 1 (Zero-Mutation Invariant)

We empirically verified **Theorem 1** ($\text{Reject} \implies S' = S_t$) under operational test vectors across $N = 500$ rejected scenarios by validating pre- and post-governance system state hashes:

$$\forall i \in \{1 \dots 500\}, \quad \text{SHA256}(S_{\text{initial}}) == \text{SHA256}(S_{\text{post-governance}}) \implies \text{Violations} :$$

•

Tested Rejection Scenarios: $N = 500$

• **State Mutation Violations:** 0

• **Status:** Verified strictly under evaluated test scenarios (Operational guarantee established in Paper 1 Theorem 1).

4. Threats to Validity

4.1 Internal Validity

All latency measurements were benchmarked within a single-threaded Python 3.13 isolated process. In multi-threaded production runtimes, inter-process communication (IPC) overhead or WASM container cold-start instantiation times may introduce additional overhead beyond

$$T_{\text{Governance Runtime}} \approx 0.008 \text{ ms}$$

4.2 External Validity

The reported unit metrics (100.00% Precision/Recall) are evaluated against the Controlled Reference Policy Dataset. While synthetic rules allow deterministic unit validation of

governance state transitions ($G(I, S) \implies S'$), real-world non-deterministic LLM prompts contain semantic noise. Schema adapters for AgentDojo and CyberSecEval were integrated to validate structural schema compatibility. Evaluation on live production agent execution traces (e.g., SWE-bench, live MCP server streams) is formally left as future work.

4.3 Construct Validity

Our evaluation prioritizes Unauthorized Execution Rate (UER), False Positive Pause Rate (FPPR), and P95 overhead latency as core performance constructs. Industrial deployments with custom compliance constraints may require additional metrics, such as GPU context switching times.

4.4 Reproducibility

To ensure scientific reproducibility, the complete reference implementation (rhizoh-runtime v0.1), dataset generator (generate_egb_dataset.py), and pure-Python evaluation suite (run_paper2_experiment.py) are made open-source without external library dependencies.

4.5 Statistical Validity

Although zero unauthorized executions were observed in the controlled reference evaluation, the reported Clopper-Pearson 95% confidence interval ($[0.00\% - 0.40\%]$) reflects the statistical uncertainty inherent in finite sample evaluation ($N = 1,000$). Evaluating larger real-world agent execution traces will further tighten this upper bound.

5. Limitations

- **No Distributed Execution:** Current benchmarks evaluate single-node runtime execution; network partition behavior remains unmodeled.
- **No GPU / VRAM Measurements:** Latency measurements focus on CPU execution bounds, omitting GPU context-switching overhead.
- **No Multi-Agent Contention:** Evaluated workloads reflect single-agent execution without concurrent multi-agent race conditions.
- **No Asynchronous Scheduler:** Runtime evaluation assumes synchronous intent interception prior to sandbox instantiation.
- **Synthetic Cohort Focus:** Benchmark evaluations rely on reference policy datasets and adapter schemas; direct evaluation on production agent execution traces is left as future work.

6. Future Work & Real-World Validation Roadmap

- **Evaluation on Production Agent Traces:** Benchmarking Rhizoh Engine against full production execution traces from SWE-bench, WebArena, and commercial agent telemetry.

- **Model Context Protocol (MCP) Integration:** Developing native MCP middleware proxies for integration with commercial LLM agent SDKs (e.g., Anthropic MCP, LangGraph, OpenAI Agents).
- **Automated $U(t)$ Weight Optimization:** Implementing dynamic Bayesian weight calibration for $w_1 \dots w_4$ under online context feedback.
- **Distributed Governance & Kubernetes Runtime:** Extending Merkle DAG provenance logs across distributed microservice nodes using Raft consensus.
- **Multi-Agent Conflict Resolution:** Formalizing game-theoretic consensus rules when multiple autonomous agents express conflicting state mutation intents simultaneously.
- **WASM Sandbox Embedding:** Direct compiled execution of $U(t)$ and OPA evaluation within WebAssembly sandboxes for edge device deployment.

7. Conclusion

Paper 2 empirically validates Rhizoh Engine. By demonstrating microsecond-scale runtime overhead ($P_{95} \leq 0.15 \text{ ms}$), $p > 0.05$ stability across scaling workloads, and empirical proof of zero-mutation state preservation under evaluated test vectors, Rhizoh demonstrates that formal epistemic governance can be efficiently integrated into LLM agent execution runtimes.